

CHAPTER 11

EXCEPTION HANDLING

Exception Handling

An exception is a problem that arises during the execution of a program. We need to handle such exceptions to prevent abrupt termination of program. The term exception means exceptional condition, it is a problem that may arise during the execution of program. A bunch of things can lead to exceptions, including programmer error, hardware failures, files that need to be opened cannot be found, resource exhaustion etc. Exception Handling is the mechanism to handle such kind of runtime malfunctions. We perform exception handling so the normal flow of the application can be maintained

Definition: An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions.

When an error occurs within method, the method creates an object and hands it off to the runtime system. The object, called an exception object, contains information about an error, including its type and the state of the program when the error occurred. Creating an exception object and handling it to the runtime system is called *throwing an exception*.

After a method throws an exception, the runtime system attempts to find something to handle it. The set of possible “something” to handle the exception is the ordered list of methods that had been called to get the method where the error occurred. The list of methods is known as the *call stack*.

Advantage of Exception Handling

- The core advantage of exception handling is that normal flow of the application is maintained.
- Separating error handling code from regular code
- It makes for clear, robust, fault-tolerant programs.

Exception normally disrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:

1. statement 1;
2. statement 2;
3. statement 3;
4. statement 4;
5. statement 5;
6. statement 6;

7. statement 7;
8. statement 8;
9. statement 9;
10. statement 10;

Suppose there is 10 statements in your program and there occurs an exception at statement 5, rest of the code will not be executed i.e. statement 6 to 10 will not run. If we perform exception handling, rest of the statements will be executed. That is why we use exception handling

Some common examples of Exception are:

- Divide by zero errors.
- Invalid input
- Opening a non-existent file.
- Heap memory exhausted.

Types of Errors

Errors may broadly classified into two categories:

1. Compile-time errors
2. Run-time errors

Compile Time Errors

All syntax errors will be detected and display by the C++ compiler and therefore these errors are known as compile-time errors. Whenever the compiler display an error, it will not create the executable file. It is therefore necessary that we fix all the errors before we can successfully compile and run the program.

Most common compile time problems are:

- i. Missing semicolons.
- ii. Missing (or mismatch of) brackets in classes and functions
- iii. Misspelling of identifier and keywords
- iv. Use of undeclared variables
- v. Incompatible types in assignments.

Run-Time Errors

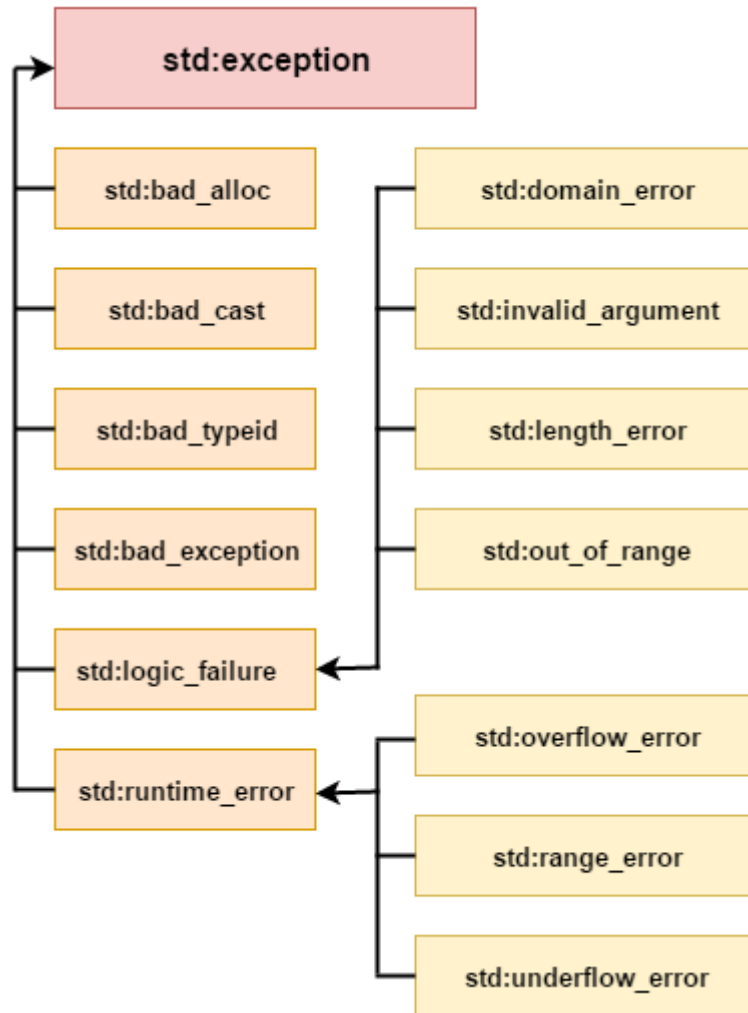
Sometimes, a program may compile successfully creating the executable file but may not run properly. Such programs may produce wrong results due to wrong logic or may terminate due to errors such as stack overflow. Most common run-time error are:

- i. Dividing an integer by zero
- ii. Trying to store a value into an array of an incompatible class or type.
- iii. Attempting to use negative size for an array.

In C++, exception is an event or object which is thrown at runtime. All exceptions are derived from **std::exception** class. It is a runtime error which can be handled. If we don't handle the exception, it prints exception message and terminates the program.

Hierarchy of Exception classes

In C++ standard exceptions are defined in `<exception>` class that we can use inside our programs. The arrangement of parent-child class hierarchy is shown below:



All the exception classes in C++ are derived from **std::exception** class.

Here is the small description of each exception mentioned in the above hierarchy –

Sr.No	Exception & Description
1	std::exception An exception and parent class of all the standard C++ exceptions.
2	std::bad_alloc This can be thrown by new .
3	std::bad_cast This can be thrown by dynamic_cast .

4	std::bad_exception This is useful device to handle unexpected exceptions in a C++ program.
5	std::bad_typeid This can be thrown by typeid .
6	std::logic_error An exception that theoretically can be detected by reading the code.
7	std::domain_error This is an exception thrown when a mathematically invalid domain is used.
8	std::invalid_argument This is thrown due to invalid arguments.
9	std::length_error This is thrown when a too big std::string is created.
10	std::out_of_range This can be thrown by the 'at' method, for example a std::vector and std::bitset<>::operator[]().
11	std::runtime_error An exception that theoretically cannot be detected by reading the code.
12	std::overflow_error This is thrown if a mathematical overflow occurs.
13	std::range_error This is occurred when you try to store a value which is out of range.
14	std::underflow_error This is thrown if a mathematical underflow occurs.

Exception Handling Keywords

C++ exception handling is built upon three keywords: **try**, **catch**, and **throw**.

- **try** – A **try** block identifies a block of code for which particular exceptions will be activated. It's followed by one or more catch blocks.

- **catch** – A program catches an exception with an exception handler at the place in a program where you want to handle the problem. The **catch** keyword indicates the catching of an exception.
- **throw** – A program throws an exception when a problem shows up. This is done using a **throw** keyword.

In C++ programming, exception handling is performed using try/catch statement. The C++ **try block** is used to place the code that may occur exception. The **catch block** is used to handle the exception.

Example1: Without try... catch.

```
#include <iostream>
using namespace std;
int main ()
{
    int i = 50;
    int j = 0;
    float k = i / j;
    cout << k << endl;
    return 0;
}
```

Output

Floating point exception (core dumped)

Example2: With try...catch block

```
#include<iostream>
using namespace std;
int main ()
{
    int i = 25;
    int j = 10;
    try
    {
        if( j == 0 )
        {
            throw "Attempted to divide by zero!";
        }

        float k = 0;
        k = i/j;
        cout << k << endl;
    }
}
```

```

        catch (const string e)
        {
            cout << e << endl;
        }

        cout<<"end of Main function";

    return 0;
}

```

Output:

Attempted to divide by zero!
end of Main function

Another example to show exception handling in C++. The output of program explains flow of execution of try/catch blocks`

Example3:

```

#include <iostream>
using namespace std;

int main()
{
    int x = -1;
    // Some code
    cout << "Before try \n";
    try {
        cout << "Inside try \n";
        if (x < 0)
        {
            throw x;
            cout << "After throw (Never executed) \n";
        }
    }
    catch (int x ) {
        cout << "Exception Caught \n";
    }

    cout << "After catch (Will be executed) \n";
    return 0;
}

```

Output:

Before try

Inside try

Exception Caught

After catch (Will be executed)

Implicit type conversion doesn't happen for primitive types. For example, in the following program 'a' is not implicitly converted to int

```
#include <iostream>
using namespace std;
```

```
int main()
{
    try {
        throw 'a';
    }
    catch (int x) {
        cout << "Caught " << x;
    }
    catch (...) {
        cout << "Default Exception\n";
    }
    cout<<"End of Main";
    return 0;
}
```

Output:

Default Exception

If an exception is thrown and not caught anywhere, the program terminates abnormally. For example, in the following program, a char is thrown, but there is no catch block to catch a char.

```
#include <iostream>
using namespace std;
```

```
int main()
{
    try {
        throw 'a';
    }
    catch (int x) {
        cout << "Caught ";
    }
    return 0;
}
```

Output:

terminate called after throwing an instance of 'char'

Handle Any Type of Exceptions (...)

If you do not know the throw **type** used in the try block, you can use the "three dots" syntax (...) inside the catch block, which will handle any type of exception: For example, in the following program, an int is thrown as an exception, but there is no catch block for int, so catch(...) block will be executed.

```
#include <iostream>
using namespace std;

int main()
{
    try {
        throw 10;
    }
    catch (char *excp) {
        cout << "Caught " << excp;
    }
    catch (...) {
        cout << "Default Exception\n";
    }
    return 0;
}
```

Output:

Default Exception

Example for Unreachable Catch block

While using multiple **catch** statements, it is important to remember that exception sub classes inside **catch** must come before catch(...) otherwise it will lead to compile time error.

```
#include<iostream>
using namespace std;
int main()
{
    try
    {
        int arr[]={1,2};
```



```

arr[2]=3/0;
throw "array index out of bound exception";
}

catch(...) //This block handles all Exception
{
cout<<"Generic exception";
}
catch(const char * e) //This block is unreachable
{
cout<<e;
}
}

```

C++ User-Defined Exceptions

The new exception can be defined by overriding and inheriting **exception** class functionality. Following steps should be follow to create a user defined function.

- In order to create a user defined exception we first included the <exception> header using the pre-processor directive. This header has the base class exception which we have to inherit and override to create any user defined exceptions
- In the next step we created a class named MyException and inherited all properties from the exception class. Then we created function what() which basically returns an error message string (in this case – C++ Exception). So whenever an exception of type MyException occurs, this message will be displayed. The return type of this function what() is char* because we are returning a character string.
- In the last step, inside the main function we create a try{}...catch() block and inside the try block we create an object of MyException class and use the throw keyword to explicitly throw an exception using this object.
- This Object is then caught in the catch block where we print out the message by accessing the what() function of this(MyException) class's object.

Let's see the simple example of user-defined exception in which **std::exception** class is used to define the exception.

Example1: (User Defined Exception)

```

#include <iostream>
#include <exception>
using namespace std;
class MyException : public exception
{
public:
    const char * what()

```

```

        {
            return "Attempted to divide by zero!\n";
        }
    };
int main()
{
    try
    {
        int x, y;
        cout << "Enter the two numbers : \n";
        cin >> x >> y;
        if (y == 0)
        {
            MyException z;
            Throw z;

        }
        else
        {
            cout << "x / y = " << x/y << endl;
        }
    }
    catch(exception e)
    {
        cout << e.what();
    }
}

```

Output:

Enter the two numbers :

10

2

x / y = 5

Enter the two numbers :

10

0

Attempted to divide by zero!

Note: In above example what() is a public method provided by the exception class. It is used to return the cause of an exception.

Example2: (User Defined Exception)

```
#include<iostream>
#include<exception>
using namespace std;
class NegativeAgeException :public exception
{
    public:
    int age;
    const char * what()
    {
        return "Age :” + age + “cannot be negative";
    }

    NegativeAgeException(int a)
    {
        age = a;
    }
};
```

```
int main()
{
    int varage;
    cout<<"Enter Age:";
    cin>> varage;
    try
    {
        if(varage < 0)
        {
            NegativeAgeException e(varage);
            throw e;

        }
        else
        {
            cout<<"Age entered is " << varage;
        }
    }
    catch(NegativeAgeException ob)
    {
        cout<<ob.what();
    }
    cout<<"rest of the statement";
    return(0);
}
```

```
}
```

Output:

Enter Age:-2

Age cannot be negative

Example3: (User Defined Exception)

```
#include <iostream>
```

```
#include <exception>
```

```
using namespace std;
```

```
class OverSpeed : public exception{
```

```
int speed;
```

```
public :
```

```
    const char* what(){
```

```
        return "check out ur car speed you are in the car not in an aeroplane ";
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    int carspeed=0;
```

```
    try
```

```
    {
```

```
        while(1)
```

```
        {
```

```
            carspeed+=10;
```

```
            if(carspeed>100)
```

```
            {
```

```
                OverSpeed s;
```

```
                throw s;
```

```
            }
```

```
            cout<<"Carspeed: "<<carspeed<<endl;
```

```
        }
```

```
    }
```

```
    catch(OverSpeed ex)
```

```
    {
```

```
        cout<<ex.what();
```

```
    }
```

```
    return 0;
```

```
}
```

Exception Handling Concept Using Classes

Example : Program to implement exception handling with single class

```
#include <iostream>
using namespace std;

class demo {
};

int main()
{
    try {
        throw demo();
    }

    catch (demo d) {
        cout << "Caught exception of demo class \n";
    }
}
```

Output:

Caught exception of demo class

Example 2: Program to implement exception handling with two class

```
#include <iostream>
using namespace std;

class demo1 {
};

class demo2 {
};

int main()
{
    for (int i = 1; i <= 2; i++) {
        try {
            if (i == 1)
                throw demo1();

            else if (i == 2)
                throw demo2();
        }

        catch(exception e) {}
        catch (demo1 d1) {
            cout << "Caught exception of demo1 class \n";
        }
    }
}
```

```

    }

    catch (demo2 d2) {
        cout << "Caught exception of demo2 class \n";
    }
}
}

```

Output:

Caught exception of demo1 class
Caught exception of demo2 class

Example 3:

Create a user defined exception for the following:

Suppose the clients have a customer_name, account_number, and balance, among other details. The deposit() and withdrawal() functions let clients to make deposits and withdrawals from their accounts, respectively. In the event that a client requests a larger withdrawal from their account than what is available, a "LowBalanceException" will take place. The client will see a notification stating, "Low Balance Exception.." in this case.

```

#include <iostream>
#include <string>
#include <exception>
Using namespace std;
class LowBalanceException : public exception
{
    public:
        const char* what()
        {
            return "Low Balance Exception..";
        }
};

class BankAccount
{
private:
    string name;
    int accountNumber;
    double balance;

public:
    BankAccount(const string& clientName, int accNumber, double initialBalance):
    name(clientName), accountNumber(accNumber), balance(initialBalance) {}

    void deposit(double amount)
    {
        balance += amount;
        cout << "Deposited: $" << amount << endl;
    }

    void withdrawal(double amount) {
        if (amount > balance) {
            throw LowBalanceException();
        } else {

```

```

        balance -= amount;
        cout << "Withdrawn: $" << amount << endl;
    }
}

void displayBalance() {
    cout << "Account Holder: " << name << endl;
    cout << "Account Number: " << accountNumber << endl;
    cout << "Balance: $" << balance << endl;
}

};

int main() {
    BankAccount client1("John Doe", 12345, 1000.0);

    client1.displayBalance();

    try {
        client1.withdrawal(1500.0);
    } catch (const LowBalanceException& e) {
        cerr << "Exception: " << e.what() << endl;
    }

    client1.displayBalance();

    return 0;
}

```

Exception handling with inheritance:

Exception handling can also be implemented with the help of inheritance. In case of inheritance object thrown by derived class is caught by the first catch block.

Example:

```

#include <iostream>
using namespace std;

class demo1 {
};

class demo2 : public demo1 {
};

int main()
{
    for (int i = 1; i <= 2; i++) {
        try {
            if (i == 1)

```

```

        throw demo1();

    else if (i == 2)
        throw demo2();
    }

    catch (demo1 d1) {
        cout << "Caught exception of demo1 class \n";
    }
    catch (demo2 d2) {
        cout << "Caught exception of demo2 class \n";
    }
}
}

```

Output:

```

Caught exception of demo1 class
Caught exception of demo1 class

```

In the above program we have made **demo2** as derived class for **demo1**. Note the **catch** block for **demo1** is written first. As **demo1** is base class for **demo2** an object thrown of **demo2** class will be handled by first **catch** block.

Exception handling with constructor:

Exception handling can also be implemented by using constructor. Though we cannot return any value from the constructor but with the help of **try** and **catch** block we can.

Example:

```

#include <iostream>
using namespace std;

class demo {
    int num;

public:
    demo(int x)
    {
        try {

            if (x == 0)
                // catch block would be called
                throw "Zero not allowed ";

            num = x;

```



```

        show();
    }

    catch (const char* exp) {
        cout << "Exception caught \n ";
        cout << exp << endl;
    }
}

void show()
{
    cout << "Num = " << num << endl;
}

};

int main()
{

    // constructor will be called
    demo(0);
    cout << "Again creating object \n";
    demo(1);
}

```

Output:

```

Exception caught
Zero not allowed
Again creating object
Num = 1

```

when **x = 0** exception is **thrown** and **catch** block is called. And when **x = 1** no exception is created.